Bug Bounty Foundations: Scope, Evidence, Reports

A friendly field guide to learning web security responsibly and preparing clear vulnerability reports

1. What is bug bounty work?

A bug bounty program invites researchers to examine specified assets under published rules. A useful researcher is careful, methodical, respectful of users, and honest about what was actually demonstrated. Participation or payment is never guaranteed.

FIRST RULE Written program scope and rules come before tools. If an asset, test, or action is not clearly allowed, stop and ask the program owner.

2. Five checks before testing

Check	Ask yourself
Asset scope	Is this exact domain, app, API, or repository listed as in scope?
Allowed methods	Are automation, scanners, authenticated testing, and particular test types permitted?
Boundaries	What is excluded, rate-limited, or prohibited?
Data safety	Could this expose, change, or delete someone else's information? Do not proceed if so.
Disclosure	Where must findings be reported, and what sharing limits apply?

3. A safe beginner practice path

1. Learn HTTP requests, responses, cookies, sessions, authentication, and access control using diagrams and official documentation.
2. Complete a beginner lab specifically provided by PortSwigger Web Security Academy. Stay within that lab's boundaries.
3. For local practice, run an intentionally vulnerable app such as OWASP Juice Shop only on your own machine or isolated training network; do not expose it publicly.
4. Use browser developer tools to observe normal requests in that training application before making any test changes.
5. Write down the expected behaviour, observed behaviour, and the smallest reproducible proof.
6. Practise remediation thinking: explain how a developer could prevent or limit the issue.

4. Example: an access-control question

Friendly example: in a deliberately designed training lab, a page appears to show a profile belonging to the currently signed-in demo user. The learning question is whether the application consistently checks who is allowed to view a record. Use only the lab's provided accounts and instructions; never try another person's real account or data.

Report section	What makes it useful
Title	Name the behaviour and affected feature in plain language.
Scope	Identify the permitted training target or in-scope asset.
Summary	Explain the access-control expectation and what differed.
Reproduction	Give minimal, repeatable steps using only permitted test accounts.
Impact	Describe realistic consequences without exaggerating.
Evidence	Include redacted screenshots or sample responses with secrets removed.
Fix idea	Recommend server-side authorisation checks and appropriate tests.

5. Report template

```
Title: [behaviour] in [feature]
Asset / lab: [exact authorised scope]
Summary: [one or two plain-language sentences]
Steps: [minimal repeatable steps in the lab]
Expected: [intended safe behaviour]
Observed: [what actually happened]
Impact: [who/what could be affected, evidence-based]
Evidence: [redacted screenshot/request/response]
Suggested fix: [defensive control]
Limitations: [what was not tested]
```

6. Stop conditions

- The asset is not explicitly in scope or the rules are unclear.
- A test would access, modify, or disclose another real user's data.
- A test could disrupt service, cause persistent changes, or create costs.
- You encounter secrets, personal data, or an unexpected production system. Stop and follow the program's disclosure guidance.
- A platform rule or local law prohibits the activity.

Trusted beginner resources

- PortSwigger Web Security Academy - free interactive labs

- OWASP Web Security Testing Guide
- OWASP Juice Shop project
- HackerOne: defining scope
- Bugcrowd: reviewing program briefs

Before you submit: final review

- The exact asset and activity were explicitly in scope at the time of testing.
- The report describes observed behaviour, not a guess based only on a tool warning.
- Reproduction uses only approved training or test accounts and has no destructive side effects.
- Evidence is minimal and redacted; no passwords, tokens, or other users' data are included.
- Impact is realistic, the suggested fix matches the cause, and limitations are stated.

A few words to know

Term	Plain-language meaning
Scope	The exact assets and activities the owner authorises.
Reproducible	Another authorised tester can follow the steps and observe the same result.
Impact	The realistic harm the issue could cause, supported by evidence.
Disclosure	How a finding is reported and what may be shared publicly.

REALITY CHECK Bug bounty is competitive. Learning, finding a valid issue, and receiving a reward are separate outcomes. A clear authorised lab portfolio is worthwhile even if a programme offers no reward.